

Design Patterns in Test Automation Frameworks (Java)

In modern test automation frameworks, applying **Design Patterns** is essential for writing clean, reusable, and scalable code. Patterns are not tied to any tool (Selenium, Appium, RestAssured, etc.) but to principles that make frameworks robust and maintainable. This guide explores the **Top 5 Design Patterns** used in Java-based test automation frameworks with explanations, code examples, practical context, and references.

■ Categories of Design Patterns

Design patterns are broadly divided into three categories. In test automation, all three categories find use cases: **Creational Patterns** - Purpose: Handle *object creation*, making systems flexible in deciding what objects to create. - Examples: Singleton (WebDriver instance), Factory (choose browser driver), Builder (test data objects). **Structural Patterns** - Purpose: Deal with *object composition*, organizing classes/objects into larger structures. - Examples: Page Object Model (POM), Decorator (extending WebDriver functionality). **Behavioral Patterns** - Purpose: Focus on *interaction between objects*, distributing responsibilities and communication. - Examples: Strategy (different waits/loggers), Observer (listeners for reporting), Template Method (base test flow). ■ Think of it like this: - Creational → How objects are *created* - Structural → How objects are *organized* - Behavioral → How objects *interact*

1. Page Object Model (POM) – Structural Pattern

Encapsulates the structure and behavior of a page into a class. Each page or component is represented as an object with locators and methods.

Context in Framework: Used in the `pages` package of the framework to model each web page or reusable component.

Examples:

- LoginPage with `login(username, password)` method
- CheckoutPage encapsulating entire order flow
- HeaderComponent (reusable component object)

Code Example:

```
public class LoginPage {
    private WebDriver driver;
    private By username = By.id("username");
    private By password = By.id("password");
    private By loginBtn = By.id("login");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void login(String user, String pass) {
        driver.findElement(username).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(loginBtn).click();
    }
}
```

```
    }  
}
```

Benefit: Removes duplicate locators, improves readability, supports reusability.

2. Singleton Pattern – Creational Pattern

Ensures only one instance of a class is created. Commonly used for WebDriver, configuration, or reporting.

Context in Framework: Used in the `core` or `driver` package to provide a single instance of WebDriver, ConfigManager, or ReportManager.

Examples:

- DriverManager for single WebDriver instance per thread
- ConfigManager for centralized configuration
- ReportManager for shared reporting object

Code Example:

```
public class DriverManager {  
    private static WebDriver driver;  
  
    private DriverManager() {} // private constructor  
  
    public static WebDriver getDriver() {  
        if (driver == null) {  
            driver = new ChromeDriver();  
        }  
        return driver;  
    }  
}
```

Benefit: Prevents resource conflicts, centralizes control, ensures consistency.

3. Factory Pattern – Creational Pattern

Abstracts object creation logic, useful when multiple variations of objects exist.

Context in Framework: Used in the `factories` package to create drivers, data providers, or API clients based on configuration.

Examples:

- DriverFactory for Chrome, Firefox, Edge, Remote
- DataProviderFactory for JSON, CSV, DB, API sources
- PageFactory (built into Selenium)

Code Example:

```
public class DriverFactory {  
    public static WebDriver getDriver(String browser) {  
        switch (browser.toLowerCase()) {  
            case "firefox": return new FirefoxDriver();  
            case "edge": return new EdgeDriver();  
        }  
    }  
}
```

```

        default: return new ChromeDriver();
    }
}

```

Benefit: Centralized object creation, easy configuration changes.

4. Strategy Pattern – Behavioral Pattern

Defines a family of interchangeable algorithms. In automation, useful for waits, assertions, or logging strategies.

Context in Framework: Used in the `utils` or `core.waits` package to implement different waiting or logging strategies.

Examples:

- Wait strategies (explicit, fluent, implicit)
- Assertion strategies (soft vs hard)
- Logging strategies (console, file, report)

Code Example:

```

interface WaitStrategy {
    void waitFor(WebDriver driver, By locator);
}

class ExplicitWait implements WaitStrategy {
    public void waitFor(WebDriver driver, By locator) {
        new WebDriverWait(driver, Duration.ofSeconds(10))
            .until(ExpectedConditions.visibilityOfElementLocated(locator));
    }
}

class FluentWaitStrategy implements WaitStrategy {
    public void waitFor(WebDriver driver, By locator) {
        new FluentWait<>(driver)
            .withTimeout(Duration.ofSeconds(10))
            .pollingEvery(Duration.ofSeconds(1))
            .until(ExpectedConditions.presenceOfElementLocated(locator));
    }
}

```

Benefit: Flexible extension of behavior without modifying core logic.

5. Builder Pattern – Creational Pattern

Used for constructing complex objects step by step, avoiding telescoping constructors.

Context in Framework: Used in the `models` or `testdata` package to build complex test objects (User, Order, API requests).

Examples:

- User or Order object creation for test data

- Building REST API requests (headers, params, body)
- Test case metadata builder (priority, owner, tags)

Code Example:

```
public class User {
    private String name;
    private String email;
    private String role;

    private User(UserBuilder builder) {
        this.name = builder.name;
        this.email = builder.email;
        this.role = builder.role;
    }

    public static class UserBuilder {
        private String name;
        private String email;
        private String role;

        public UserBuilder setName(String name) { this.name = name; return this; }
        public UserBuilder setEmail(String email) { this.email = email; return this; }
        public UserBuilder setRole(String role) { this.role = role; return this; }
        public User build() { return new User(this); }
    }
}

// usage
User user = new User.UserBuilder().setName("Alice").setRole("Admin").build();
```

Benefit: Readable, maintainable test data setup.

■ References & Further Reading

- Gamma, Helm, Johnson, Vlissides - 'Design Patterns: Elements of Reusable Object-Oriented Software'
- SeleniumHQ: <https://www.selenium.dev/>
- TestNG Listeners: <https://testng.org/doc/documentation-main.html>
- Martin Fowler - Patterns of Enterprise Application Architecture
- Refactoring Guru (great for pattern illustrations): <https://refactoring.guru/design-patterns>